Prompts y práctica de requerimientos con IA

Material para actividad por equipos

1. Práctica del tema Ingeniería de Software

Nombre de la práctica:
Práctica 1 - Definición de requerimientos del proyecto final con apoyo de la IA.

Actividad:
Cada equipo trabajará sobre su propuesta de proyecto y generará su primer documento formal de requerimientos.

2. Objetivo de la práctica

Que cada equipo transforme una idea inicial en un documento claro de análisis de requerimientos, usando la IA como apoyo y aplicando revisión humana para validar, corregir y justificar lo generado.

3. Roles sugeridos

Rol	Responsabilidad
Analista	Redacta problema, usuario objetivo, necesidad, alcance y fuera del alcance.
Operador de IA	Ejecuta prompts, registra respuestas relevantes y documenta decisiones.
Validador / tester	Revisa claridad, ambigüedades, verificabilidad y criterios de aceptación.
Documentador	Integra el documento final y cuida formato, consistencia y bitácora.

4. Prompt base para la IA

```
Actúa como analista de requerimientos de software.

Estamos desarrollando una utilería o manejador llamado: [NOMBRE DEL PROYECTO].

Idea inicial:
[DESCRIBIR IDEA]

Ayúdanos a definir:
1. Problema que resuelve.
2. Usuario objetivo.
3. Alcance del proyecto.
4. Funcionalidades principales.
5. Requerimientos funcionales.
6. Requerimientos no funcionales.
7. Historias de usuario.
8. Criterios de aceptación.
9. Riesgos o ambigüedades.

No generes código. Solo ayúdanos con análisis de requerimientos.
```

5. Prompts específicos por fase

5.1 Refinar el problema

```
Analiza la siguiente idea de proyecto y ayúdanos a convertirla en un problema claro.

Idea:
[PEGAR IDEA]

Entrega:
1. Problema principal.
2. Necesidad que se busca atender.
3. Usuario afectado.
4. Consecuencias de no resolver el problema.
5. Redacción mejorada del planteamiento del problema.
```

5.2 Definir alcance y fuera del alcance

```
Con base en el problema del proyecto, ayúdanos a definir el alcance.

Entrega una tabla con:
1. Funcionalidades dentro del alcance.
2. Funcionalidades fuera del alcance.
3. Justificación de cada exclusión.

El proyecto debe ser viable para un total de de 8 semanas, trabajando en equipos de 3 integrantes.
```

5.3 Generar requerimientos funcionales

```
Genera requerimientos funcionales para el proyecto.

Reglas:
- Numéralos como RF-01, RF-02, RF-03.
- Cada requerimiento debe ser claro y verificable.
- Evita requerimientos demasiado generales.
- Incluye funcionalidades normales y manejo de errores.
- No propongas funcionalidades fuera del alcance definido.
```

5.4 Generar requerimientos no funcionales

```
Genera requerimientos no funcionales para el proyecto.

Considera:
1. Usabilidad.
2. Rendimiento.
3. Seguridad básica.
4. Mantenibilidad.
5. Modularidad.
6. Compatibilidad.
7. Pruebas.
8. Documentación.

Redáctalos como RNF-01, RNF-02, RNF-03 y procura que puedan verificarse.
```

5.5 Crear historias de usuario

```
Con base en los requerimientos funcionales, genera historias de usuario.

Usa el formato:
Como [tipo de usuario], quiero [funcionalidad], para [beneficio].

Entrega las historias de usuario necesarias y relaciona cada una con el requerimiento funcional correspondiente.
```

5.6 Crear criterios de aceptación

```
Para cada historia de usuario, genera criterios de aceptación.

Usa un formato claro:
Dado [contexto], cuando [acción], entonces [resultado esperado].

Incluye criterios para:
- Caso exitoso.
- Datos incorrectos.
- Errores esperados.
- Casos límite.
```

5.7 Revisar ambigüedades

```
Revisa los siguientes requerimientos y detecta problemas.

Evalúa:
1. Ambigüedad.
2. Falta de verificabilidad.
3. Requerimientos duplicados.
4. Requerimientos demasiado grandes.
5. Funcionalidades fuera del alcance.
6. Riesgos técnicos.

Entrega una lista de observaciones y recomendaciones, pero no reescribas todo todavía.
```

6. Producto a entregar

Crear un documento en word convertido a PDF, que contenga:

- Nombre de la empresa cliente
- Nombre de la empresa desarrolladora
- Nombre del proyecto.
- Tabla de contenido (índice formal)
- Integrantes y roles.
- Problema que resuelve.
- Usuario objetivo.
- Alcance.
- Fuera del alcance.
- Requerimientos funcionales.
- Requerimientos no funcionales.
- Historias de usuario.
- Criterios de aceptación.

- Riesgos o ambigüedades.
- Bitácora de uso de la IA utilizada.

7. Plantilla de documento de requerimientos

```
Nombre de la empresa cliente:
Nombre de la empresa desarrolladora:
Nombre del proyecto:
Integrantes:
Roles:

1. Planteamiento del problema
[Redactar el problema de forma clara]

2. Usuario objetivo
[Describir quién usará o se beneficiará de la solución]

3. Alcance del proyecto
[Indicar qué funcionalidades sí se desarrollarán]

4. Fuera del alcance
[Indicar qué no se desarrollará y por qué]

5. Requerimientos funcionales
RF-01:
RF-02:
RF-03:

6. Requerimientos no funcionales
RNF-01:
RNF-02:
RNF-03:

7. Historias de usuario
HU-01:
HU-02:
HU-03:

8. Criterios de aceptación
CA-01:
CA-02:
CA-03:

9. Riesgos o ambigüedades
[Listar dudas, riesgos técnicos o puntos por validar]

10. Uso de Codex
[Resumen de prompts, decisiones y correcciones]
```

8. Plantilla de bitácora de Codex

Fase	Prompt usado	Resultado de la IA	Qué aceptamos	Qué modificamos	Responsable
Análisis					
Alcance					
Requerimientos funcionales					
Requerimientos no funcionales					
Historias de usuario					
Criterios de aceptación					

9. Preguntas guía para revisión del equipo

- ¿El problema está claramente definido?
- ¿El usuario objetivo es específico?
- ¿Los requerimientos son verificables?
- ¿Hay requerimientos demasiado grandes o vagos?
- ¿Los criterios de aceptación permiten probar la funcionalidad?
- ¿Codex propuso algo que el equipo decidió rechazar?
- ¿Qué falta aclarar antes de diseñar el sistema?

10. Errores comunes a evitar

Error	Ejemplo	Corrección
Requerimiento vago	El sistema debe ser fácil de usar.	El sistema debe mostrar mensajes claros de error e instrucciones de ejecución.
Alcance demasiado grande	Crear un sistema escolar completo.	Crear una utilería para procesar archivos CSV de calificaciones.
Confundir solución con problema	Hacer una app web con login.	Los docentes necesitan automatizar reportes.
No considerar errores	Solo pensar en archivos válidos.	Incluir archivos vacíos, columnas faltantes y rutas incorrectas.
Historias sin beneficio	Como usuario quiero cargar archivo.	Como docente quiero cargar archivo para generar reportes automáticamente.

11. Tarea para la siguiente semana

- Versión corregida del documento de requerimientos.
- Mínimo 5 historias de usuario, considera las necesarias para cubir los requerimientos funcionales
- Criterios de aceptación por historia.
- Matriz simple de trazabilidad: requerimiento, historia de usuario, criterio de aceptación y prueba sugerida.

Requerimiento	Historia de usuario	Criterio de aceptación	Prueba sugerida
RF-01 Cargar archivo CSV	Como docente quiero cargar un CSV	Si el archivo existe, se procesa correctamente	Probar con archivo válido
RF-02 Validar columnas	Como docente quiero saber si faltan columnas	Si falta una columna, se muestra error	Probar CSV incompleto

Un proyecto no fracasa únicamente porque el código esté mal. Muchas veces fracasa porque nadie entendió bien qué debía construirse. Por eso esta semana no se busca programar: primero se debe definir correctamente el problema, el alcance y los requerimientos.